Python Quiz 4 - Coding Practice

Coding Practice on Files - Student Copy

Instructions:

- Complete both functions in each practice question.
- Your functions must return the required results and must not contain `print()` statements.
- Assume the files exist, contain exactly the data shown, and have no blank lines.

Practice 1 - Guided Tour Booking

A travel company offers several guided tours. Each tour has a limited number of available places and a price per person.

Each record contains:

- the name of the tour;
- the number of available places, recorded as an integer; and
- the price per person, recorded as a decimal number.

The file `tours.txt` contains the following data, separated by the `"!"` symbol:

```
TOUR!PLACES!PRICE
River Walk!12!18.50
Heritage Trail!8!24.00
Garden Visit!15!16.00
Night Tour!6!28.50
```

You must complete the two functions below. Your functions must **return** the required results and must not contain any `print()` statements.

Function A: Reading the tour information

Complete the function: `read_tours(filename)`

The function takes one parameter:

- `filename` (type: str): the name of the file to read.

The function must:

1. Read the specified file.
2. Return a list of tuples in the same order as the records in the file.
   Each tuple must have this structure: `(tour_name, available_places, price_per_person)`

The available places must be stored as an **integer**, and the price per person must be stored as a **float**.

For the file shown above, the function should return:

```
[
    ('River Walk', 12, 18.5),
    ('Heritage Trail', 8, 24.0),
    ('Garden Visit', 15, 16.0),
    ('Night Tour', 6, 28.5)
]
```

Function B: Finding suitable tours

Complete the function: `find_suitable_tours(filename, group_size, maximum_budget)`

The function takes three parameters:

- `filename` (type: str): the name of the tour file;
- `group_size` (type: int): the number of people who want to join the tour;
- `maximum_budget` (type: float): the maximum total amount that the group can spend.

For each tour, calculate the total cost as follows: `total_cost = price_per_person * group_size`

A tour is suitable if both of the following conditions are satisfied:

- the number of available places is greater than or equal to `group_size` and
- the total cost is **less than or equal to** `maximum_budget`.

The function must:

1. Call the read `tours()` function written in Function A.
2. Select all tours that **satisfy both requirements**.
3. **Return a new list** of tuples in the same order as the records in the file.

Each tuple must have this structure: `(tour_name, total_cost)`

If no tour satisfies both requirements, the function must return the empty list `[]`.

TODO

- Complete the missing code in the two function definitions.
- Do not change the given main program provided below the two functions.

```
def read_tours(filename):
    pass









def find_suitable_tours(filename, group_size, maximum_budget):
    pass









# DO NOT CHANGE THE MAIN CODE BELOW:
filename = "tours.txt"
suitable_tours = find_suitable_tours(filename, 4, 80.0)
print(suitable_tours)
```

Expected output

```
[('River Walk', 74.0), ('Garden Visit', 64.0)]
```

Practice 2 - Plant Growth Monitoring

A community garden records the initial and final heights of several plants during a growing period. All heights are measured in centimetres.

Each record contains:

- the name of the plant;
- the initial height, recorded as a decimal number; and
- the final height, recorded as a decimal number.

The file `plant_growth.txt` contains the following data, separated by the "`**`" symbol:

```
PLANT**INITIAL**FINAL
Basil**12.5**18.0
Mint**15.0**18.5
Rosemary**20.0**27.5
Thyme**10.5**12.0
Lavender**18.0**24.0
```

You must complete the two functions below. Your functions must **return** the required results and must not contain any `print()` statements.

Function A: Calculating plant growth

Complete the function: `read_plant_growth(filename)`

The function takes one parameter:

- `filename` (type: str): the name of the file to read.

For each plant, calculate its growth as follows: `growth = final_height - initial_height`

The function must:

1. Read the specified file.
2. Calculate the growth of each plant.
3. Return a list of tuples in the same order as the records in the file.

Each tuple must have this structure: `(plant_name, growth)`

For the file shown above, the function should return:

```
[
    ('Basil', 5.5),
    ('Mint', 3.5),
    ('Rosemary', 7.5),
    ('Thyme', 1.5),
    ('Lavender', 6.0)
]
```

Function B: Finding fast-growing plants

Complete the function: `find_fast_growing_plants(filename, minimum_growth)`

The function takes two parameters:

- `filename` (type: str): the name of the plant growth file;
- `minimum_growth` (type: float): the minimum required growth.

A plant is selected if its growth is **greater than or equal to** `minimum_growth`.

For each selected plant, calculate the extra growth as follows:

```
extra_growth = growth - minimum_growth
```

The function must:

1. Call the `read_plant_growth()` function written in Function A.
2. Select all plants whose growth is **greater than or equal** to `minimum_growth`.
3. Calculate the **extra growth** for each selected plant.
4. **Return a new list** of tuples in the same order as the records in the file.

Each tuple must have this structure: `(plant_name, extra_growth)`

If no plant satisfies the requirement, the function must return the empty list [].

TODO

- Complete the missing code in the two function definitions.
- Do not change the given main program provided below the two functions.

```
def read_plant_growth(filename):
    pass




def find_fast_growing_plants(filename, minimum_growth):
    pass




# DO NOT CHANGE THE MAIN CODE BELOW:
filename = "plant_growth.txt"
plants = find_fast_growing_plants(filename, 5.0)
print(plants)
```

Expected output

```
[('Basil', 0.5), ('Rosemary', 2.5), ('Lavender', 1.0)]
```

Practice 3 - Recycling Collection Review

A recycling programme records the weights of paper and plastic collected at several collection sites. All weights are recorded in kilograms.

Each record contains:

- the name of the collection site;
- the weight of paper collected, recorded as an integer; and
- the weight of plastic collected, recorded as an integer.

The file `recycling.txt` contains the following data, separated by the `"%"` symbol:

```
SITE%PAPER%PLASTIC
Market%42%35
Library%28%51
Park%24%19
Community Club%55%48
Sports Centre%31%36
```

For every site in this file, the paper and plastic weights are different. Therefore, there are no ties.

You must complete the two functions below. Your functions must **return** the required results and must not contain any `print()` statements.

Function A: Reading the recycling data

Complete the function: `read_recycling_data(filename)`

The function takes one parameter:

- `filename` (type: str): the name of the file to read.

The function must:

1. Read the specified file.
2. Return a list of tuples in the same order as the records in the file.

Each tuple must have this structure: `(site_name, paper_weight, plastic_weight)`

The paper and plastic weights must be stored as **integers**.

For the file shown above, the function should return:

```
[
    ('Market', 42, 35),
    ('Library', 28, 51),
    ('Park', 24, 19),
    ('Community Club', 55, 48),
    ('Sports Centre', 31, 36)
]
```

Function B: Finding major collection sites

Complete the function: `find_major_collection_sites(filename, minimum_weight)`

The function takes two parameters:

- `filename` (type: str): the name of the recycling data file;
- `minimum_weight` (type: int): the minimum paper or plastic weight required for a site to be selected.

A site is selected if at least one of the following conditions is satisfied:

- the paper weight is greater than or equal to `minimum_weight` or
- the plastic weight is greater than or equal to `minimum_weight`.

For each selected site:

- Use "Paper" as the leading material if the paper weight is higher. Otherwise, use "Plastic".
- Calculate the total collected weight: total_weight = paper_weight + plastic_weight.

The function must:

1. Call the `read_recycling_data()` function written in Function A.
2. Select all sites that satisfy **at least one of the two** `minimum-weight` conditions.
3. Determine the **leading material** and **calculate the total weight** for each selected site.
4. **Return a new list** of tuples in the same order as the records in the file.

Each tuple must have this structure: `(site_name, leading_material, total_weight)`

If no site satisfies either requirement, the function must return the empty list `[]`.

TODO

- Complete the missing code in the two function definitions.
- Do not change the given main program provided below the two functions.

```
def read_recycling_data(filename):
    pass









def find_major_collection_sites(filename, minimum_weight):
    pass









# DO NOT CHANGE THE MAIN CODE BELOW:
filename = "recycling.txt"
sites = find_major_collection_sites(filename, 40)
print(sites)
```

Expected output

```
[('Market', 'Paper', 77), ('Library', 'Plastic', 79),
 ('Community Club', 'Paper', 103)]
```